

CODEBASE & ARCHITECTURE REVIEW · 14 JUL 2026

EduBase SaaS — where the build actually stands

A read of the repository against your own planning documents (*Complete Modules & Features v2*, *ER Diagram v2*) — what's real, what's scaffolding, and what to fix before the next module.

2 modules functionally complete of 17 planned	~26 / 62 database tables actually migrated	54 / 89 model files are empty placeholders	76 / 82 controllers are empty placeholders	0 / 5 mobile apps contain any code
--	---	---	---	---

The short version

The two documents you shared describe an ambitious full rebuild: 17 modules, 140+ features, 62 tables, three institution types (school/college/university), a subscription-gated module system, AI reporting, WhatsApp, an online payment gateway, and five companion mobile apps. That's the target, not the current state — and that's completely normal for where this project is.

What's actually in the repository today is a genuine, working **vertical slice**: tenant-scoped auth, campus/session/class/section setup, student admission and directory, fee plans and challan generation/payment, and double-entry GL accounting — all built in Livewire 3 on Laravel 11 (composer.json says ^11.0, not 13 — worth correcting in the docs). Everything outside that slice — Exam & Attendance, HRM, House Management, Semester/GPA, Hostel, Library, Online Admissions, AI, WhatsApp, payment gateways, and every mobile app — exists only as **empty placeholder files**: a folder structure and file names that match the spec, with zero

bytes inside. Only one commit (fdb6314) generated that entire skeleton at once; the second commit (415e308) is where real work happened, on the slice above.

The good news: the slice that *is* built is architecturally sound in its core idea (global-scope multi-tenancy, Livewire component structure, GL modeling). The bad news: there's an in-progress permissions migration that will break the app on a fresh migrate, plus two authorization holes that let any logged-in tenant user escalate their own privileges. Those need to be closed **before** starting the next module — they're small, contained fixes, not a rewrite.

What's genuinely working

Built, wired to routes, and backed by real migrations — this is your foundation. Keep building on top of it.

Multi-tenancy core

A

`HasTenant`

trait + global

`TenantScope`

auto-filters every tenant model by session `tenant_id`. The old bug (Tenant model self-scoping, duplicate User model) from your last review is fixed.

`app/Models/Traits/HasTenant.php`

Student admission & directory

432-line admission form, profile view, searchable/exportable directory (Excel export via maatwebsite), media library for photos/documents.

`app/Livewire/StudentAdmission.php`

GL accounting

Chart of accounts, GL groups/accounts, journal entries, general ledger view, accounting periods — genuine double-entry modeling.

Academic structure

Campus → Session → Class → Section →

Campus-Class mapping, full CRUD via Livewire, real relationships, scopes for active records.

`app/Livewire/{Campus,Class,Session,CampusClass}Manager`

Fee plans & challans

Fee plan authoring, campus-class mapping, challan generation, payment collection, print/PDF output (dompdf), a 3-part bank challan layout.

`app/Models/Finance/{Challan,FeePlan,...}.php`

Roles & permissions UI

Working CRUD for roles/permissions and user assignment — currently sitting on the legacy schema (see Critical Issues: this is mid-migration

```
app/Livewire/{ChartOfAccounts,GLAccounts,JournalEn to Spatie).  
app/Livewire/{ManageRoles,UsersAndPermissions}.php
```

Fix before the next module

Ranked by how much damage they'd do left alone. The first two are quick fixes; do them before writing new feature code so you're not building on top of a break.

CRITICAL

The new permissions migration will break the app on the next fresh migrate

You deleted the custom roles/permissions migrations and added Spatie's `laravel-permission` migration — but Spatie's tables use its own schema (`id`, `name`, `guard_name`) and drop the columns your live UI writes to (`tenant_id`, `slug`, `description`). `ManageRoles::saveRole()` will throw a SQL error the moment someone runs migrations fresh. `App\Models\User` also doesn't use Spatie's `HasRoles` trait yet, so the two systems aren't actually connected — this is a half-finished cutover, not a working integration.

Decide: finish the Spatie cutover (add `HasRoles` to `User`, migrate existing role data, update `ManageRoles` to use Spatie's API) or roll it back until you're ready.

Either way, don't leave it half-applied. Ref:

[database/migrations/2026_05_05_213400_create_permission_tables.php](#) ,
[app/Livewire/ManageRoles.php:95-102](#)

CRITICAL

Any logged-in user can create accounts and grant themselves any role

`UsersAndPermissions::save()` and `ManageRoles::saveRole()/updateRolePermissions()` have no authorization check at all — only auth + tenant middleware guard the route. A teacher or clerk account that can reach `/users-and-permissions` can create a new user and assign it a super-admin-equivalent role.

Add an explicit `$this->authorize(...)` or role check at the top of both methods. Ref: [app/Livewire/UsersAndPermissions.php:58](#) ,
[app/Livewire/ManageRoles.php:79,91](#)

HIGH**Tenant isolation silently turns off outside a normal web request**

TenantScope only adds its `WHERE tenant_id` clause when a tenant id is present in session/config. In a queued job, scheduled command, or artisan tinker session — none of which have a session — the scope applies *no filter*, meaning any tenant-scoped query returns every tenant's rows. This will matter the moment you add background jobs (salary calculation, SMS blasts, scheduled reports all appear in your module list).

Make the fail mode explicit: either throw when no tenant context is resolvable, or require jobs/commands to explicitly set tenant context. Ref:

[app/Scopes/TenantScope.php:19-26](#) ,
[app/Models/Traits/HasTenant.php:14-23](#)

HIGH**Challan print/download has no permission check**

FeePolicy exists with a correct tenant-ownership check, but it's never called — `PrintChallanController::bulkPrint/downloadPdf` let any authenticated tenant user bulk-print or download fee/payment documents with zero authorization.

Wire `FeePolicy` into the controller, or add an inline permission check consistent with the rest of the fee module. Ref:

[app/Http/Controllers/PrintChallanController.php:12-64](#)

MEDIUM**Dead duplicate models shadow the real ones**

`app/Models/Fee/*` and `app/Models/Accounting/*` are empty leftover namespaces from an earlier refactor — the working code actually lives in `app/Models/Finance/*`. Same class names, different folders: an easy wrong-import mistake waiting to happen for future-you or a teammate.

Delete the empty `Fee/` and `Accounting/` folders, plus `Academic/ClassModel.php` (dead duplicate of `SchoolClass.php`).

LOW**Repo hygiene: scratch scripts and unpinned dependencies**

`assign_role.php` at the project root calls Spatie methods that don't exist on your User model yet (would throw if run).

`scratch_indexes.php/scratch_indexes_families.php` are dev one-offs. `spatie/laravel-permission` and `spatie/laravel-medialibrary` are both pinned to "*" in `composer.json`, so a routine `composer update` could pull a breaking major version unnoticed.

Remove the three root scripts (or move to a `scripts/` folder, gitignored). Pin

Module-by-module status

Against your own 17-module documentation. "Not started" means the model/controller files exist as 0-byte placeholders and no migration backs them — pure scaffolding, not a bug.

#	MODULE	STATUS	WHAT EXISTS TODAY
1	Dashboard	NOT STARTED	Component renders a static view only — no KPI widgets, no charts
2	Admissions & Fee Management	MOSTLY COMPLETE	Admission, fee plans, challan generation/payment/print all working. Family-wise payment, installment challans, discount workflow, and the reports (defaulters, day-wise, class-wise) still need verification/finishing
3	Extras (Student Tools)	~10%	Student directory + Excel export only. No reminders, certificates, QR codes, bulk import, promote/transfer, roll-no slips, alumni handling
4	Finance (GL/Accounting)	MOSTLY COMPLETE	Chart of accounts, journal entries, general ledger all real. Trial balance, P&L, balance sheet, cash report, voided-transactions view not found
5	Exam & Attendance	NOT STARTED	Zero migrations, zero controller/model code
6	HRM	NOT STARTED	Zero migrations, zero controller/model code
7	General & House Management	NOT STARTED	No trace anywhere in the codebase
8–12	Settings modules	PARTIAL	Core setup (campus/class/session/fee) is real; no dedicated exam-settings, HRM-

			settings, SMS/WhatsApp/subscription settings UI
13	Semester & GPA	NOT STARTED	College/university only — zero code
14	Hostel Management	NOT STARTED	College/university only — zero code
15	Library Management	NOT STARTED	College/university only — zero code
16	Online Admissions Portal	NOT STARTED	College/university only — zero code
17	SaaS extras (AI, WhatsApp, payment gateway, module gating, mobile apps)	NOT STARTED	Config files are empty, no SDKs installed, all 5 mobile app folders contain only an empty package.json each

Suggested working order

You said you want to go module by module — here's a sequence that stabilizes the foundation first, then finishes what's closest to done before opening new ground. Each phase assumes the previous one is actually closed out, tested, and committed.

O

STABILIZE — DO THIS FIRST
Close the four Critical/High issues above

Resolve the permissions migration collision, add the missing authorization checks, decide the tenant-isolation fail mode, and wire FeePolicy into the challan controller. This is a day or two of focused work, not a new module — but every module after this inherits these risks if left alone.

1

FINISH, DON'T START
Close out Module 2 (Admissions & Fee) and Module 4 (Finance/GL)

These are your most mature modules. Finish the missing pieces — admission queries/CRM, family-wise payment, installment challans, challan discounts, fee-defaulter and day-wise reports, plus trial balance / P&L / balance sheet / cash report on the GL side. This gets two whole modules to "done" rather than five

modules stuck at 60%.

2

NEXT MODULE

Module 3 — Extras (Student Tools)

Certificates, QR codes, bulk student import, batch promote/transfer, roll-no slips. Natural next step since it extends the student data you already have, and unblocks the "not currently enrolled / alumni" fee workflows also listed under Module 2.

3

CORE OPERATIONAL MODULE

Module 5 — Exam & Attendance

Zero code today but core to daily school operation. Build attendance first (it's simpler and everything else — SMS alerts, house management, parent app — depends on it), then exam marks/reporting.

4

CORE OPERATIONAL MODULE

Module 6 — HRM

Employee records, attendance, salary plans and payroll. Independent enough from the student-facing modules that it can be built in parallel once you have spare capacity, but sequence it after Exam/Attendance if working solo.

5

DECISION POINT

Reassess scope: school-only v1, or all three institution types?

Everything built so far only serves the School (K-12) type. Modules 13–16 (Semester/GPA, Hostel, Library, Online Admissions) only apply to college/university tenants and are 100% unbuilt. Before investing in them, it's worth explicitly deciding whether v1 ships as school-only with those modules deferred — that alone would cut your remaining scope by roughly a third.

6

DEFER TO LAST

SaaS differentiators — AI, WhatsApp, payment gateway, module gating, mobile apps

These depend on the core modules existing first (AI reports need attendance/fee data; the payment gateway needs a finished challan flow; module gating only matters once there's more than one gated module). Tackle these once the